

Mal Unified Payments Platform

Architecture and migration strategy

Deliverable 2, Senior Data Engineering Take-Home

April 28, 2026

Executive summary. Mal’s three payment squads (Cards, Transfers, Bill Payments) ship into three schemas, three databases, and three sets of squad-specific reports. Finance reconciliation, Customer 360, and CBUAE corridor reporting all suffer. Two known reconciliation incidents have been traced to float rounding in Cards. We propose one canonical event-grain Parquet store, quarantine-pattern validation, and a 24-week, four-phase migration that lands each squad with zero blast radius in Phase 0 and a mechanical cutover by Phase 2. D1 is a working 449-line implementation with 24 tests passing. D3 is the on-call data-quality dashboard. The biggest risk is adoption, not technology; Phase 0 is what de-risks it.

Context

Mal is a UAE neobank with Islamic banking products. Three squads own three payment surfaces: **Cards** (auth/capture lifecycle, MCC-tagged merchants, mostly AED with some USD/GBP), **Transfers** (domestic plus expat-driven remittance corridors to India, Philippines, Pakistan, Bangladesh, Egypt), and **Bill Payments** (utilities, telecom, government, scheduled vs executed). Each squad ships into its own database; there is no cross-product event store today.

Stakeholder	Pain today
Finance reconciliation	Three pipelines, manual joins, two known float rounding incidents in Cards
Customer Success	No cross-product customer view; <code>customer_id</code> semantics differ across two of three squads
Compliance, CBUAE reporting	Corridor volume sits in Transfers; failure-code vocabulary differs from Cards; no shared audit trail
Platform reliability	No single dashboard for “all payment failures last 7 days”; each squad has its own status vocabulary

Target architecture

Five load-bearing decisions, each a direct response to a real failure mode.

- 1. Event-grain, not payment-grain.** auth, capture, and refund are one logical payment but multiple events linked by `correlation_id`. The auth-to-settle latency query in D1 (SQL 03) cannot be written cleanly on squad-grained data.
- 2. Integer minor units.** `amount_minor` parsed at the squad boundary using `decimal.Decimal` with banker’s rounding (`ROUND_HALF_EVEN`). The v1 to v2 migration in `schema/migrations.py` is the recovery path for v1 data already in flight.
- 3. raw_payload preserved per row.** Squads do not lose information when they hand data to the platform. This is an adoption lever, not a technical one. Auditability comes for free.
- 4. payment_method_details as a JSON bag.** When the Wallet squad ships in Q3, we add an adapter, not a schema migration. Bounds the cost of a new payment type to one file.
- 5. Quarantine, not fail-fast.** Bad rows from one squad never block another. Quarantine files preserve the original payload plus the validation error. Pipeline exits zero even with quarantined records; non-zero is reserved for infrastructure failures.

Target architecture: one canonical event model

Five layers. Adapters absorb squad-specific quirks. Pydantic v2 validates. Bad rows quarantine without blocking. The canonical Parquet store is the single source of truth.

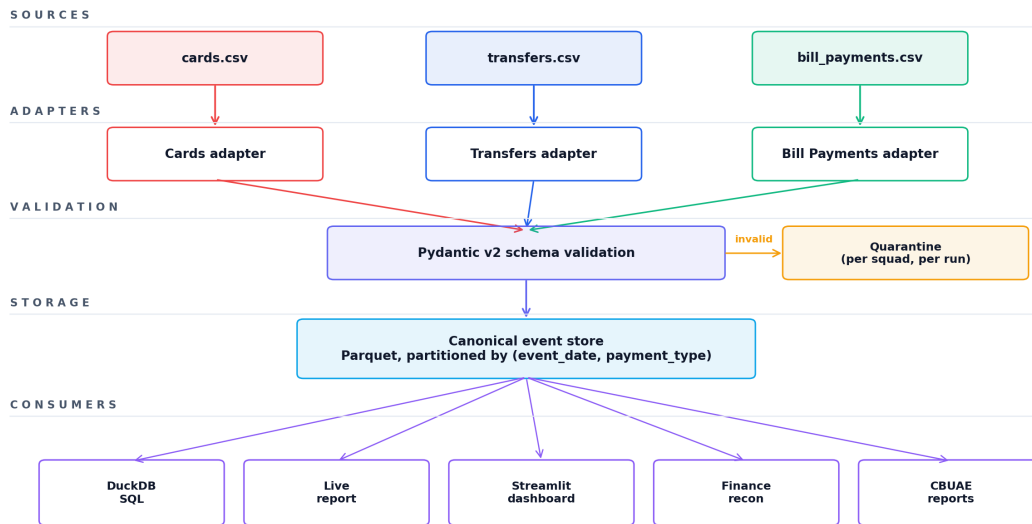


Figure 1: Target architecture: five layers, one canonical store

Migration timeline: 24 weeks across four phases

Phase 0 has zero blast radius. Phases 1 to 2 are mechanical once the delta report is green for two weeks. Phase 3 is opt-in and runs at each squad's pace.

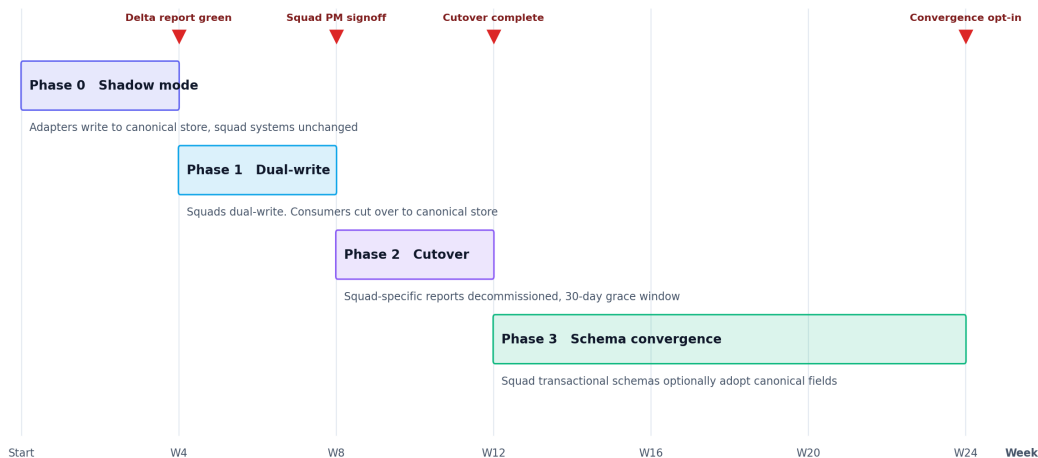


Figure 2: Migration timeline: 24 weeks across four phases

Migration strategy

Phase 0, shadow mode (weeks 1 to 4). Adapters read squad data and write to the canonical store. Squad systems unchanged. Daily delta report; zero blast radius. Exit when delta report is green for two consecutive weeks and the squad PM signs off on the failure-code mapping.

Phase 1, dual-write (weeks 5 to 8). Squads dual-write via the adapter. Finance and Customer Success cut over to read from the canonical store. Squad-specific dashboards keep working. Exit when both consumers accept canonical as primary.

Phase 2, cutover (weeks 9 to 12). Squad-specific reports decommissioned with a 30-day grace window. CBUAE reports run off canonical. Squads keep their transactional databases; canonical is the analytics source of truth.

Phase 3, schema convergence (weeks 13 to 24). Optional. Where the squad PM agrees, transactional schemas

start adopting canonical fields. The platform does not block on this. The v1 to v2 migration in D1 is the template for any future schema bump: defaults for legacy rows, banker's rounding on numerics, every output row revalidated, source data never mutated, idempotent on rerun.

Production readiness

D1 was scoped for a 30-second local run. At Mal's expected production volume of around 100K transactions per day, the following gaps matter.

Table 2: Production gap punch list

Gap	What D1 does	What production needs	Priority
Orchestration	One Python process	Dagster: asset-based DAG, SQL sensors, lineage UI for CBUAE audit	High
Ingestion	CSV files in <code>data/raw/</code>	Debezium plus Kafka for CDC, 30-second freshness SLO, replay support	High
Customer identity	<code>customer_id</code> trusted from each squad	Identity service deduplicating across phone, email, national ID	High
PII tokenization	<code>last4</code> only, no real PII in mocks	Vault or KMS for tokens; analytics never sees raw PII	High
Observability	One INFO line per squad per run	Structured logging, per-stage metrics, alerts on quarantine spikes	High
Schema registry	Pydantic models in the repo	Confluent or Git-based registry with evolution rules	Medium
Multi-region DR	Single local Parquet directory	UAE-resident primary, second region for DR (CBUAE residency)	Medium
Backfill, replay	Re-run <code>make seed</code> and <code>make run</code>	Replay-from-Kafka with idempotent writes, Dagster backfill jobs	Medium
Dashboards	GitHub Pages report, Streamlit (D3)	Looker or Superset on the canonical store, squad-specific saved views	Medium
FX rates	Hardcoded USD, AED, GBP	Price-service integration with timestamped rates and audit trail	Medium

The High items are non-negotiable for a production launch. Medium items can land iteratively after the canonical store is live. None of them change the canonical model.

Risk register

Risk	Likelihood	Impact	Mitigation
Squad PM deprioritizes Phase 0 onboarding	Medium	High	Phase 0 has zero blast radius; squad does no work; we run the adapter against their export
Vocabulary mapping disputes between squads	Medium	Medium	Mapping owned by platform team; disputes escalate to one Director
Customer identity collisions across squads	High	High	Identity service is a Phase 1 dependency, not a Phase 0 blocker

Risk	Likelihood	Impact	Mitigation
CBUAE residency requirements bite mid-rollout	Low	High	UAE-resident object store from day one; DR region designed in, not retrofit
Adapter coverage gap in production	Medium	Medium	Quarantine pattern catches it; daily quarantine-rate alert on D3 catches the spike

Success metrics

Adoption (lagging). Three squads on canonical by week 12; 100% finance recon; 80% squad reports retired. **Quality (leading).** Quarantine under 2% per squad; zero unannounced schema-drift per quarter; recon delta under 0.01% by row, exact by amount; surfaced live on D3. **Performance.** Freshness 30s p95; daily backfill under 10 min at 100K txn/day; 99.9% monthly availability.